

Hands-on Lab Slides: TinyML

Hands-on Lab: TinyML in Action : Build Your First Gesture Recognition Model

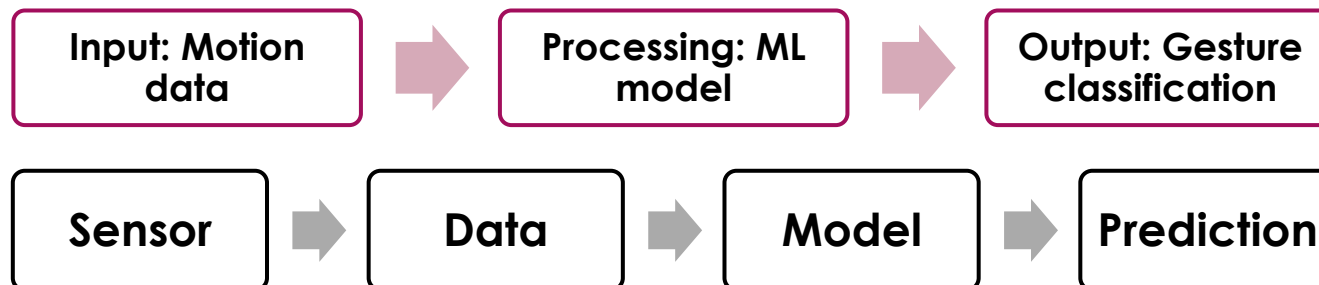
By the end of this lab, you will

- Simulate sensor data
- Train a lightweight ML model
- Apply TinyML constraints
- Perform real-time inference

Problem Statement : Build a Gesture Recognition System

- Input: Motion data from Accelerometer (X, Y, Z)
- Processing: ML model
- Output: Gesture classification (left, right, up, down)

TinyML System Overview



Hands-on Lab Slides: TinyML

Step 1 – Data Simulation

- Generate synthetic accelerometer data
- Represent different gestures
- Create labelled dataset

Step 2 - Data Visualization

- Plot X vs Y values
- Observe pattern separation
- Validate data quality

Step 3 – Train-Test Split

- 80% training
- 20% testing
- Prevent overfitting

Step 4 – Feature Scaling

- Normalize input values
- Improve model performance
- Ensure consistency

Step 5 – Model Training

Models used:

- Decision Tree (lightweight)
- Small Neural Network

Model Evaluation

Compare:

- Accuracy
- Model complexity
- Suitability for edge

Step 6 – Model Evaluation

- Accuracy
- Precision
- Recall

Step 7 – Simulating TinyML Constraints

- Reduce precision (quantization)
- Observe performance impact
- Optimize model

Step 8 – Real-Time Inference

- Input live sample
- Run prediction
- Output gesture

Hands-on Lab Slides: TinyML

TinyML Mindset

- ⚡ Smaller models
- ⚡ Faster execution
- ⚡ Lower power
- "In TinyML, efficiency is more important than complexity."

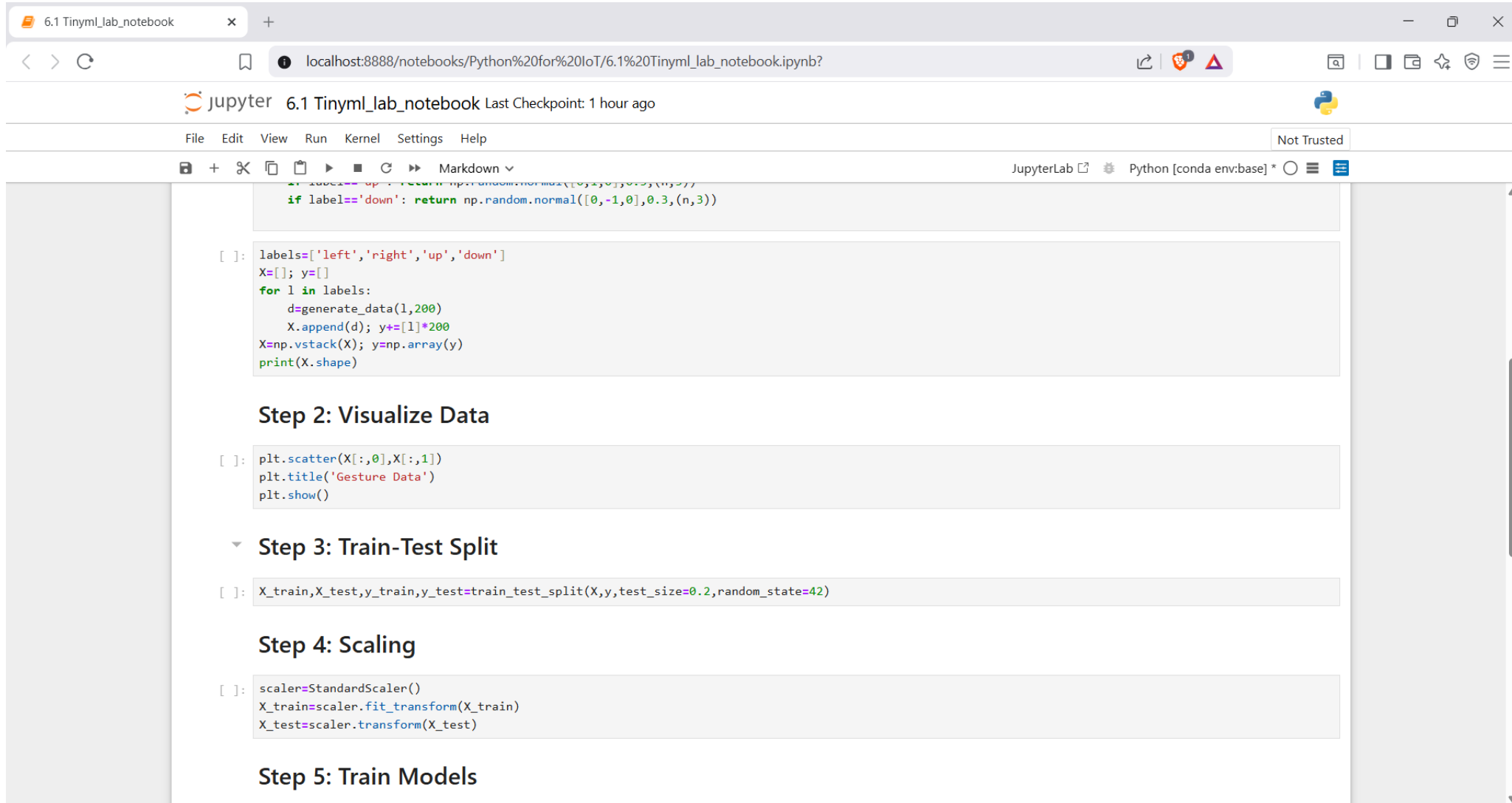
Real Device Mapping

- Simulation → Real Device
- Jupyter → Arduino / ESP32
- Model → TFLite
- Prediction → Embedded system

Common Challenges

- Low accuracy → Improve data
- Memory issues → Reduce model size
- Noise → Better preprocessing
- "These are the same challenges you'll face in real-world TinyML projects."

Hands-on Lab Slides: TinyML



The screenshot shows a JupyterLab interface with a notebook titled "6.1 Tinyml_lab_notebook". The browser address bar shows the URL: `localhost:8888/notebooks/Python%20for%20IoT/6.1%20Tinyml_lab_notebook.ipynb?`. The notebook has a "Not Trusted" warning. The code is organized into sections:

```
def generate_data(label):  
    if label=='up': return np.random.normal([0,1,0],0.5,(n,3))  
    if label=='down': return np.random.normal([0,-1,0],0.3,(n,3))  
  
[ ]: labels=['left','right','up','down']  
X=[]; y=[]  
for l in labels:  
    d=generate_data(l,200)  
    X.append(d); y+=l*200  
X=np.vstack(X); y=np.array(y)  
print(X.shape)
```

Step 2: Visualize Data

```
[ ]: plt.scatter(X[:,0],X[:,1])  
plt.title('Gesture Data')  
plt.show()
```

Step 3: Train-Test Split

```
[ ]: X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

Step 4: Scaling

```
[ ]: scaler=StandardScaler()  
X_train=scaler.fit_transform(X_train)  
X_test=scaler.transform(X_test)
```

Step 5: Train Models

Hands-on Lab Slides: TinyML

Mini Challenge

- Try adding:
 - New gestures
 - More data
 - Different model
- “This is your opportunity to experiment and improve your model.”

Key Takeaways

- ✓ TinyML pipeline
 - ✓ Lightweight models
 - ✓ Optimization importance
 - ✓ Real-time inference
- You’ve now experienced the complete TinyML workflow—from data to inference.